Universidad Nacional de Loja



Carrera de Computación
Asignatura: Estructura de Datos

Informe Técnico Avance 4

Grupo 01

Integrantes:
Carlos Martinez
Emily Salas
Selena Castillo
Elian Jimenez
Cristhian Davila

Docente:
Ing. Cristian
Narvaéz

Fecha: 30/10/2026

2026-2027

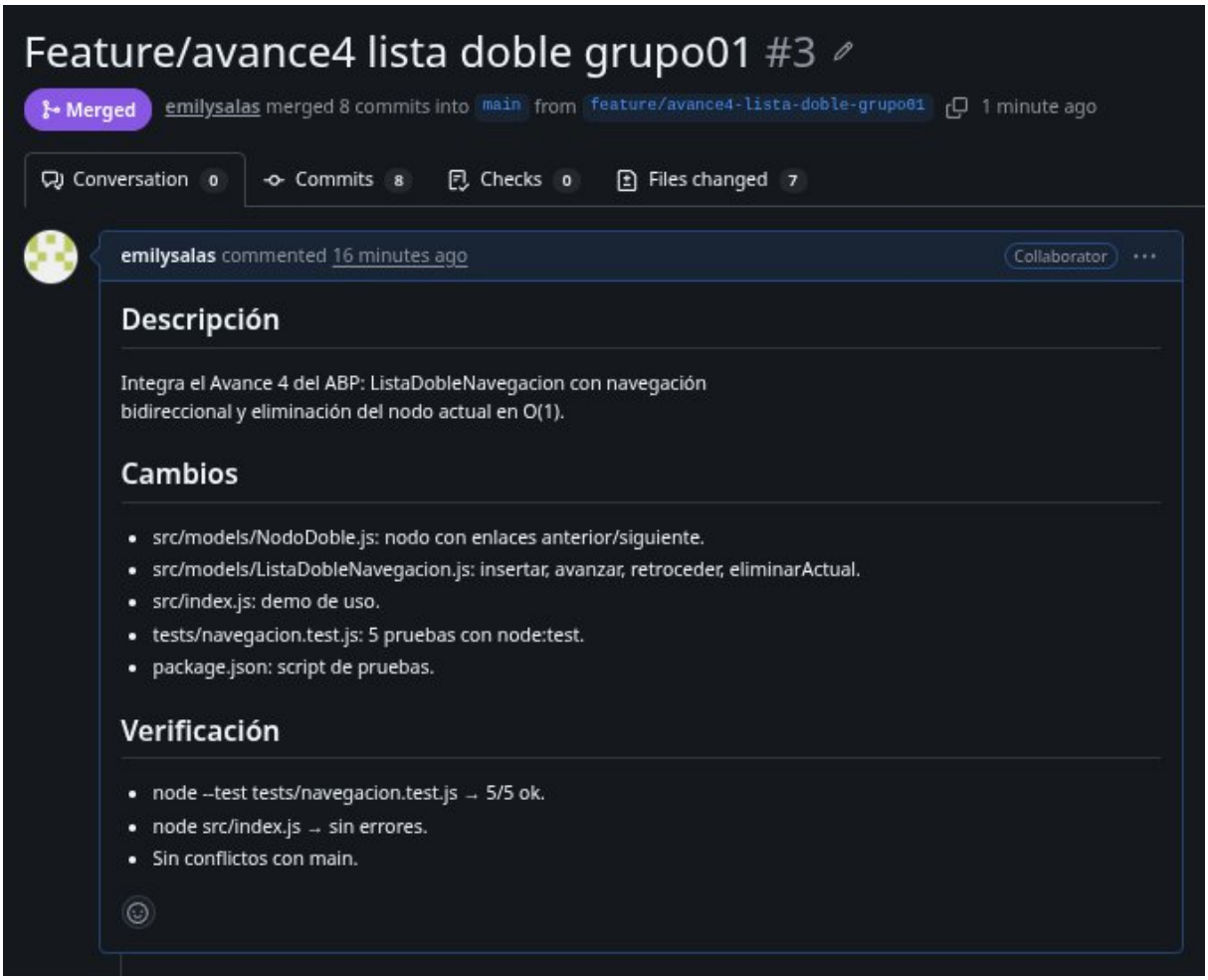
1. Enlace al repositorio y evidencia Git

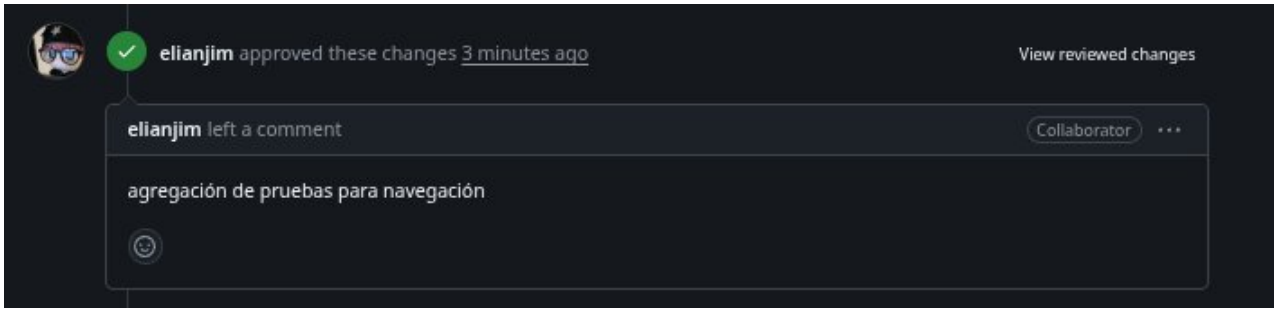
El código del Avance 4 se encuentra en el repositorio del grupo:
https://github.com/carlosamartinezcarrion/ABP-quadTree-zamora-monitor.git

Historial de commits del avance

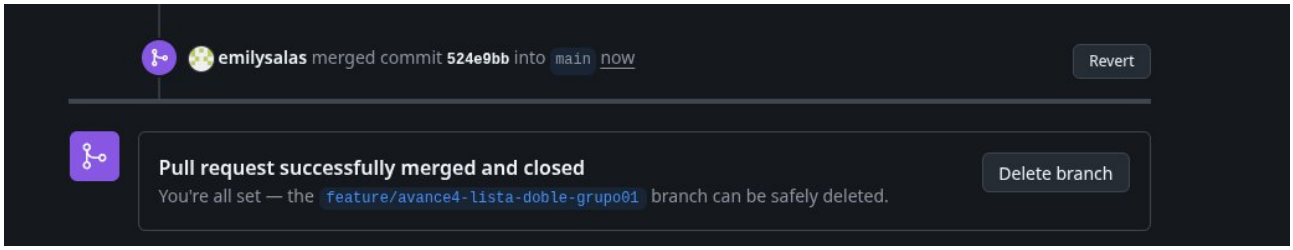


Pull Request aprobado





Commit Hash del merge a la rama principal

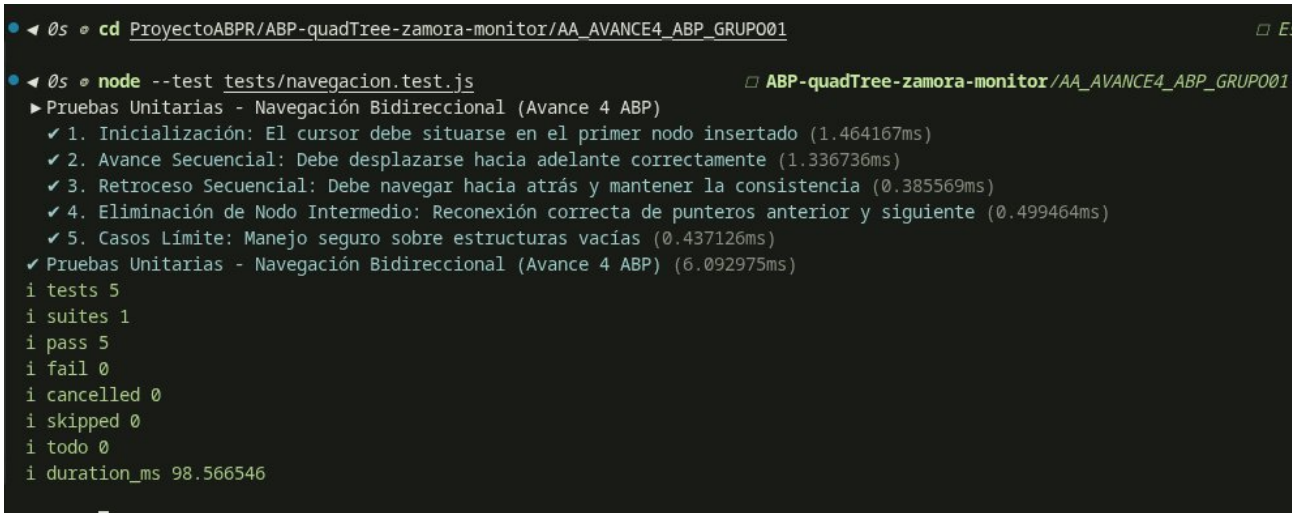


2. Matriz de cobertura de pruebas

Las pruebas se ejecutaron con el ejecutor nativo de Node.js sobre tests/navegacion.test.js. Las cinco pruebas terminaron en estado **ok**, sin fallos, cancelaciones ni omisiones.

Prueba	Método(s) validado(s)	Qué verifica	Resultado
Inicialización	insertar(), obtenerActual()	El cursor queda en el primer nodo (id 101) y la longitud es 3.	**ok**
Avance secuencial	avanzar()	Recorre 102 y 103; al llegar a la cola devuelve null.	**ok**
Retroceso secuencial	retroceder()	Vuelve a 102 y 101; en la cabeza devuelve null.	**ok**
Eliminación de nodo intermedio	eliminarActual(), retroceder()	Reconecta anterior y siguiente; la longitud baja a 2 y el cursor pasa a 103.	**ok**
Casos límite	Los cuatro métodos	Lista vacía: obtenerActual(), avanzar() y retroceder() devuelven null; eliminarActual() devuelve false.	**ok**

Captura de la consola



Salida de node --test: las cinco pruebas aparecen con la marca de aprobado (pass 5, fail 0).

3. Análisis de complejidad temporal y espacial

La estructura es una lista doblemente enlazada que conserva los punteros cabeza, cola y actual (el cursor). Como cada nodo guarda sus referencias anterior y siguiente, y el cursor señala directamente el nodo de trabajo, ninguno de los tres métodos necesita recorrer la lista. Sea *n* la cantidad de nodos almacenados.

Método avanzar()

```
if (this.actual &&
  this.actual.siguiente){ this.actual =
  this.actual.siguiente; return
  this.actual.dato;
  }
```

Complejidad temporal: O(1). El método comprueba si existe un nodo siguiente, mueve el cursor a esa referencia y devuelve su dato. Son una comparación y una asignación, sin ciclos; el costo es el mismo con 3 nodos que con 3 millones, incluso cuando el cursor ya está en la cola y se devuelve *null*.

Complejidad espacial: O(1). No crea nodos ni estructuras auxiliares; solo reasigna actual.

Método retroceder()

```
if (this.actual &&
  this.actual.anterior){ this.actual =
  this.actual.anterior; return
  this.actual.dato;
  }
```

Complejidad temporal: O(1). Es el caso simétrico de avanzar(). Gracias al puntero anteriorse llega al nodo previo de forma directa, sin recorrer la lista desde la cabeza (que en una lista simple costaría O(n)).

Complejidad espacial: O(1). Solo se modifica la referencia del cursor.

Método eliminarActual()

```
if(!this.actual)returnfalse;
enlaza anterior.siguiente con siguiente (o actualizacabeza)
enlaza siguiente.anterior con anterior (o actualiza cola)
actual = siguiente, o anterior si era la cola
nodo.siguiente=nodo.anterior=null;longitud--;
```

Complejidad temporal: O(1). Al conocer el nodo a eliminar mediante el cursor, basta con reconectar sus dos vecinos y, si era la cabeza o la cola, actualizar esos punteros. Todos los casos (nodo intermedio, extremos, único nodo, lista vacía) ejecutan una cantidad fija de asignaciones.

Complejidad espacial: O(1). Solo se usa una referencia temporal al nodo eliminado, cuyos enlaces se ponen en *null* para que el recolector de basura libere la memoria.

Resumen

Método	Tiempo	Espacio adicional
avanzar()	O(1)	O(1)
retroceder()	O(1)	O(1)
eliminarActual()	O(1)	O(1)

La estructura completa ocupa O(*n*) por los *n* nodos que almacena, pero ninguno de estos métodos añade memoria que crezca con *n.*

4. Declaración de ética e inteligencia artificial

Uso de asistentes de IA

Se utilizó el asistente Gemini como apoyo en la corrección y organización de los códigos, corrigiendo errores de ejecución y explicación de como crear las ramas con sus debidos pull y push.

Prompts utilizados

1. ¨Ayudame, el siguiente código me presenta errores que deberia hacer:...¨
2. ¨Explícame el código, su funcionamiento y proceso...¨
3. ¨Como puedo subir mis cambios a la siguiente rama de mi repositorio del git hub¨

La IA no sustituyó el trabajo de implementación; el Grupo 1 comprende el funcionamiento del código y el análisis presentado.